README

SafeVault Capstone Project - Microsoft Security and Authentication

Overview

SafeVault is an ASP.NET Core MVC security-demonstration application developed in **Visual Studio 2026** and targeting **.NET 10**. The course did not specify a required development environment. Visual Studio 2026 was selected as a developer workflow choice; the project remains a standard .NET/ASP.NET Core solution that can also be built and tested with the .NET CLI.

This guide is intended to make peer review straightforward whether the reviewer normally uses Visual Studio or Visual Studio Code. SafeVault demonstrates input validation, parameterized SQLite access, password hashing, cookie authentication, role-based access control (RBAC), antiforgery protection, and automated security testing.

> **Final verified status:** The known-good project builds without errors and the automated suite completes with **21 tests passed, 0 failed**.

1. Prerequisites

Before reviewing the project, install:

- **.NET 10 SDK**
- **Git** if cloning from the command line
- One of the following development environments:
  - **Visual Studio 2026** with the **ASP.NET and web development** workload
  - **Visual Studio Code** with **C# Dev Kit**
  - another environment capable of running the .NET CLI

The project targets .NET 10. Reviewers should use a compatible .NET 10 SDK rather than changing the project's target framework.

2. Download the Project from GitHub

After the repository is published, clone the repository URL supplied with the peer-review submission.

Git command line

```
git clone <SafeVault repository URL>
cd SafeVault
```

GitHub Desktop

1. Open GitHub Desktop.
2. Choose the SafeVault repository from the clone dialog.
3. Select a local destination and clone the repository.

3. Open the Project

Visual Studio 2026

1. Open Visual Studio.
2. Select **Open a project or solution**.
3. Open the SafeVault solution provided in the repository.
4. Allow NuGet package restore to complete before building.

Visual Studio Code

1. Open VS Code.
2. Select **File > Open Folder** and choose the cloned SafeVault repository folder.
3. Open an integrated terminal at the solution/repository root.
4. Restore the project dependencies:

```
dotnet restore
```

> **Reviewer note:** Visual Studio 2026 is not required to inspect the source or use the normal .NET CLI build/test workflow. A reviewer who is more comfortable with VS Code can use the commands below.

4. Build and Run SafeVault

Visual Studio 2026

1. Select **Build > Rebuild Solution**.
2. Confirm that the solution builds without errors.
3. Run the SafeVault web project with the IDE's normal Start/Debug command.
4. Use the localhost address displayed for the active launch profile.

Visual Studio Code / .NET CLI

From the repository root:

```
dotnet restore
dotnet build
```

Run the SafeVault web project with the .NET CLI. If `SafeVault` is the relative project path in the cloned repository:

```
dotnet run --project SafeVault
```

If the local clone uses a different relative path, point `dotnet run --project` to the provided SafeVault web project's `.csproj` file.

Use the localhost URL printed by the running application. Do not assume a fixed port because local launch configuration can differ between machines.

5. Run the Automated Security Tests

Visual Studio 2026

1. Open **Test Explorer**.
2. Select **Run All Tests**.
3. The final known-good project contains **21 passing tests**.

Visual Studio Code / .NET CLI

From the solution/repository root:

```
dotnet test
```

The suite covers input validation, password hashing and verification, unsuccessful and successful login behavior, RBAC, protected endpoints, and HTTP-level SQL-injection/XSS-style attack scenarios.

6. Authentication Test Accounts and Secrets

SafeVault uses development-only account seeding during development, but development credentials are intentionally kept outside source control. A GitHub reviewer should **not** expect plaintext development passwords or a populated User Secrets store to be included in the repository.

> **Security note:** Do not add development passwords, `secrets.json`, or other credentials to the GitHub repository. The automated tests provide reproducible evidence of the implemented authentication and authorization behavior without requiring secret material to be published.

7. Security Features to Review

- **Input validation:** Data Annotation validation and an allowlisted username field.
- **SQL injection prevention:** user-controlled database values are supplied through parameterized SQLite commands.
- **Password security:** passwords are hashed before database storage.
- **Authentication:** cookie-based sign-in establishes an authenticated identity.
- **RBAC:** the Admin controller requires the `admin` role.
- **CSRF protection:** POST forms use antiforgery protection.
- **XSS protection:** restrictive validation is combined with normal Razor output handling rather than deliberately rendering submitted data as raw HTML.
- **Security testing:** the final suite contains **21 passing unit/integration tests**.

8. Visual Studio 2026 vs. Visual Studio Code

The project was authored and debugged in Visual Studio 2026, but use of that IDE was a developer choice rather than a Capstone requirement. The source is intended to remain reviewable using standard .NET tooling.

Visual Studio provides integrated solution, build, debugging, NuGet, and test tooling. A VS Code reviewer can perform the core reproducible build and test operations with the .NET CLI commands shown in this guide.

Why Visual Studio 2026 was used: The project author has long-standing familiarity with Visual Studio and selected it for a predictable build/debug workflow after experiencing build and compile friction with VS Code on earlier projects.

9. Common Review Issues

NuGet packages need restoration

```
dotnet restore
```

Confirm the installed SDK

```
dotnet --info
```

Confirm that a compatible .NET 10 SDK is installed.

HTTPS development certificate

Local browser testing may require the review machine's normal .NET development-certificate configuration. This is a local environment issue, not an application credential requirement.

Localhost port differs

Use the URL displayed by the running application or the active local launch settings. Do not review the project against an assumed hard-coded port.

10. Additional Project Documentation

See **`SafeVault_Capstone_Summary.md`** for the full project summary, including:

- the three Capstone activities;
- vulnerabilities investigated;
- protections and hardening applied;
- authentication and RBAC implementation;
- testing strategy and final results;
- how Microsoft Copilot assisted with development and debugging.

Peer-Review Quick Check

- ☐ Restore and build the solution.
- ☐ Run the automated tests and confirm the expected test suite completes.
- ☐ Review parameterized SQLite access and input-validation rules.
- ☐ Review authentication and RBAC implementation.
- ☐ Review the Activity 3 SQL-injection/XSS attack tests.
- ☐ Read `SafeVault_Capstone_Summary.md`.

SafeVault Capstone Project - reviewer setup guide